

Relatório do mini e-commerce distribuído

O projeto foi desenvolvido com Python na versão 3.13.9 , utilizando FastAPI. SGBD foi o PostgreSQL. Toda a infraestrutura de apoio (bancos de dados e message broker) foi isolada dentro do Docker Desktop usando docker-compose. Assim, evitando a necessidade de instalar múltiplos softwares ou bibliotecas na máquina local, garantindo um ambiente mais limpo e uma inicialização rápida.

Estrutura das APIs e comunicação entre os serviços

- API Gateway (Porta 8000): É a única porta visível para o cliente externo. Ela recebe todas as requisições e faz o redirecionamento interno para as APIs corretas, além de aplicar o monitoramento de saúde do sistema.
- Serviço de Usuários (Porta 5001): Cuida do cadastro e login dos clientes. As senhas não são salvas com texto puro por questões de segurança, foi utilizado o Bcrypt para transformá-las em códigos criptografados (hash) antes de salvar no banco PostgreSQL.
- Serviço de Produtos (Porta 5002): Gerencia o catálogo do e-commerce. Possui um validador de tokens para garantir que apenas administradores cadastrem produtos.
- Serviço de Pedidos (Porta 5003): Registra as intenções de compra relacionando o ID do usuário ao ID do produto.

O fluxo principal utiliza chamadas HTTP/REST síncronas, onde o gateway atua como um intermediário direto repassando os dados com a biblioteca httpx. Mas para o serviço de pedidos, implementei uma comunicação assíncrona baseada usando RabbitMQ. Quando um pedido é feito, o serviço salva a informação localmente e dispara um evento imediato para uma fila interna do RabbitMQ. Isso simula um cenário real onde o pedido é aceito na hora e o processamento de estoque ou pagamento acontece em segundo plano.

Para o serviço de produtos, existem duas cópias idênticas do banco de dados (products_db_1 e products_db_2).

Para gerenciar isso, quando um novo produto é cadastrado por um POST, o código abre uma transação com ambas as instâncias do PostgreSQL ao mesmo tempo. O sistema só devolve uma mensagem de sucesso ao usuário se o produto for gravado com sucesso nos dois bancos. Se um deles falhar ou estiver indisponível, a operação sofre um rollback completo nos dois para evitar dados desalinhados.

Para as consultas GET /products, o sistema utiliza o algoritmo de round-robin. As requisições de leitura são distribuídas alternadamente entre os dois bancos (uma chamada lê do banco 1, a próxima lê do banco 2 e por aí em diante), para equilibrar o tráfego.

A segurança do ecossistema é baseada em tokens JWT.

Quando um usuário faz login com sucesso na porta 5001, a API gera uma chave digital contendo informações do usuário (userId, email e sua função no sistema: user ou admin). Esse token é assinado com uma senha secreta interna.

A partir daí, qualquer ação protegida (como cadastrar um produto ou fechar um pedido) exige que esse token seja enviado no cabeçalho HTTP da requisição (authorization: Bearer). A API gateway recebe esse token do cliente, repassa intacto para o serviço interno de destino e o respectivo serviço faz a checagem. Como apenas administradores têm o privilégio admin gravado no payload assinado do token, o sistema bloqueia automaticamente qualquer tentativa de um usuário comum criar produtos, devolvendo um erro 403 sem sequer tocar no banco de dados.

Durante os testes, para evitar a necessidade de manipulação manual das tabelas diretamente pelo terminal ou utilitários de banco, criamos uma regra dinâmica no código de registro. Se o e-mail cadastrado for **admin@teste.com**, o sistema concede o cargo de admin automaticamente, para facilitar a validação

Para garantir a integridade dos serviços do e-commerce, a API gateway executa uma tarefa de 5 em 5 segundos para verificar se os serviços estão “vivos”, a partir de um método GET (mas que na verdade não está requisitando nada).

Se algum serviço (como o de pedidos) cair e não responder por duas vezes seguidas, o Gateway marca esse serviço como "inativo" e gera um alerta no painel de logs (pode ser verificado no CMD). Se um cliente tentar usar uma rota daquele serviço caído, o gateway não deixa a requisição travada esperando até dar um erro genérico de tela branca; ele intercepta imediatamente o pedido e devolve uma mensagem com o erro 503 (serviço indisponível). Quando o serviço volta a rodar, a API Gateway (que vai estar rodando aquele script de checar a cada 5 secs), vai atualizar o status para "ativo" e o sistema do serviço vai voltar a funcionar sozinho.

Dessa maneira se apenas 1 serviço cair, e os outros não, os ativos ainda assim poderão trabalhar funcionalmente.